Docker :

Docker fonctionne sur un model client - serveur.
Le deamon docker (serveur) est le moteur qui créé exécute et gère les conteneurs, le client lui est l'interface qu'on utilise pour envoyer des commandes au deamon

Image : un modèle en lecture seule "un plan" qui contient tout ce dont une application à besoin pour s'exécuter, (code, bibliothèque, etc)

Conteneur : instance vivante et exécutable d'une image, isolé de l'hôte et des autres conteneurs

L'objectif principal est d'exploiter des configurations de docker mal sécurisés pour obtenir des privilèges plus élevés sur le system hôte.
Si on appartient au group docker on peu comme vu précédement lancer un conteneur qui a un accès complet au system de fichiers de l'hôte

Socket Docker Accessible : si le fichier docker.sock (généralement dans /var/run/docker.sock) est le point de communication avec le deamon docker si il est accessible en écriture on peux lancer des nouveaux conteneurs privilégiés et prendre le contrôle de l'hôte.

Repertoires Partagés (Volumes) : Si le System de fichier racine de l'hôte (/) ou un autre repertoir sensible est monté dans un conteneur, on pourras evidemment accéder à ces volumes, potentiellement récupérer des clés SSH et autres joyeusetés...

Kubernetes :

Kubernetes alias K8S est un platforme open-source qui sert à automatiser le déploiment, et la gestion d'applications conteneurs
Il fonctionne avec un noeud maître (le control panel) qui prend les décisions et des noeuds travailleurs (worker nodes) qui éxécutent les applications dans des conteneurs
Pods: La plus petite unité de travail dans Kubernetes, un pod contient un ou plusieurs conteneurs possède sa propre adresse IP et son propre nom d'hôte, fonctionne comme une petite machine virtuelle

Des mauvaises configuration comme un acces anonyme à l'API peuvent avoir de lourdes conséquences pour interroger l'API le meilleur outil actuel est kubernetctl qu'on peu trouver sur gitHub. La possibilité de créer un pod peux entraîner l'accès aux fichiers hôtes comme avec docker, encore une fois kubernetctl "automatise" très bien ça.

Logrotate :

Logrotate est un utilitaire standart de Linux conçu pour gerer les fichiers de journaux (logs). Il effectue automatiquement la rotation, la compression et suppressions des anciens fichiers de logs pour éviter qu'ils consomment de l'espace disk inutile
Si un user dispose de droits d'écriture sur un fichier de logs et que logrotate est configuré pour s'exécuter en root sur cette application certaines versions de logrotate vulnérables peuvent alors permettrent une privesc.

Techniques Diverses :

Capture de Trafic Passive : On peu bien sur laisser tourner une ecoute passive du reseau bien que très rare les protocoles en texte clair existent encore
Privileges NFS Faibles : un partage reseau (NFS) mal configuré, avec no_root_squash, cet option permet a un user root de créé des fichiers qui appartiendront a root du serveur NFS, on peu en abuser pour créé un program qui une fois exécuté sur la cible nous donnera un shell root.